

Uncertain systems control project

Anna Coscetti

1 Introduction

The objective of this work is the design of a controller for an AGV system represented in Figure 1, according to various synthesis techniques, characterized by uncertain parameters. The synthesis techniques employed are LQG and H_∞ synthesis. After analyzing their robustness, a robust controller was developed using DK-iteration.

The initial part of this project includes the selection of parameters and validation of the dynamic model, while the second part covers the design of the controllers and the corresponding analyses.

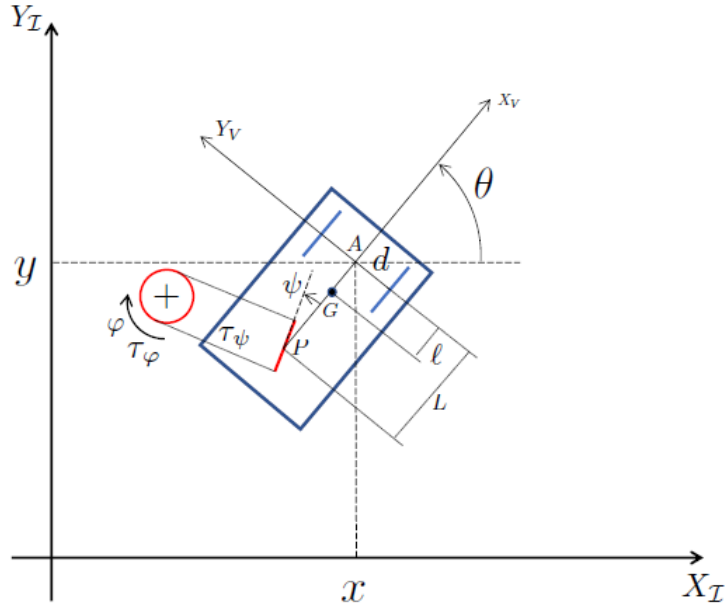


Figure 1: Automated Guided Vehicle (AGV)

2 Parameter selection and system dynamics

The following table (Table 1) contains the parameters with their respective uncertainties:

Parameter	Nominal value	Uncertainty 1	Uncertainty 2
ℓ	0.4	—	$\pm 5\%$
L	1	—	—
d	0.3	—	—
r_p	0.3	$\pm 5\%$	—
r_A	0.3	$\pm 5\%$	—
M_V	15	—	—
m_p	0.5	—	—
m_A	0.5	—	—
I_{GZ}	0.655	—	$\pm 10\%$
I_{PZ}	0.23	—	$\pm 10\%$
I_{PY}	0.15	—	—
I_{AY}	0.12	—	—
I_{AZ}	0.8	—	—
c_φ	0.01	—	—
c_ψ	0.02	—	—
K_{m_1}	1.08	$\pm 10\%$	$\pm 10\%$
T_{m_1}	0.005	$\pm 20\%$	$\pm 20\%$
K_{m_2}	0.335	$\pm 10\%$	$\pm 10\%$
T_{m_2}	0.002	$\pm 20\%$	$\pm 20\%$
T_1	$T_{m_1}/2$	$\pm 5\%$	$\pm 5\%$

Table 1: Nominal values and parametric uncertainties

On the system, and more precisely on the rear motor-wheel (the driving one), two torques act: τ_φ (rolling torque, which allows the wheel to translate) and τ_ψ (steering torque, which enables the vehicle's maneuverability by acting on the steering angle of the wheel itself).

The equations governing the dynamics and kinematics of the AGV are:

$$\dot{x} = r_p \dot{\varphi} \cos \theta \cos \psi$$

$$\dot{y} = r_p \dot{\varphi} \sin \theta \cos \psi$$

$$\dot{\theta} = -\frac{r_p}{L} \dot{\varphi} \sin \psi$$

$$\tau_\varphi = \left(ar_p^2 \cos^2 \psi + b \frac{r_p^2}{L^2} \sin^2 \psi + I_{PY} \right) \ddot{\varphi} - I_{PZ} \frac{r_p}{L} \dot{\psi} \sin \psi + \left(b \frac{r_p^2}{L^2} - ar_p^2 \right) \dot{\varphi} \dot{\psi} \sin \psi \cos \psi + c_\varphi \dot{\varphi}$$

$$\tau_\psi = -I_{PZ} \frac{r_p}{L} \dot{\varphi} \sin \psi + I_{PZ} \ddot{\psi} - I_{PZ} \frac{r_p}{L} \dot{\varphi} \dot{\psi} \cos \psi + c_\psi \dot{\psi}$$

where

$$a = \left(M_V + m_p + 2m_A + 2 \frac{I_{AY}}{r_A^2} \right)$$

$$b = \frac{1}{2} \left(M_V \ell^2 + m_p L^2 + 2m_A d^2 + I_{GZ} + I_{PZ} + 2I_{AZ} + 2I_{AY} \frac{d^2}{r_A^2} \right)$$

and where c_φ and c_ψ are linear friction coefficients acting on the rolling and steering axes respectively.

The two actuators are also characterized by the following nominal transfer functions:

$$\bar{G}_{\tau_\varphi} = \frac{\bar{K}_{m2}}{(T_{m2}s+1)(T_{m2}/100s+1)}$$

$$\bar{G}_{\tau_\psi} = \frac{\bar{K}_{m1}e^{-T_1s}}{(T_{m1}s+1)}$$

3 Task and linearized system

As the task objective, the motion of the AGV was imposed along the x -axis at a constant speed $v_x = 1$ m/s and subsequently linearized the system around this dynamic equilibrium. The matrices of the dynamic system in state-space form are therefore:

$$A = \begin{bmatrix} 0 & v_x & 0 & 0 & 0 \\ 0 & 0 & 0 & \frac{-v_x}{L} & 0 \\ 0 & 0 & \frac{-c_\varphi}{aR_p^2 + I_{PY}} & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & \frac{-(c_\psi - (I_{PZ}v_x)/L)}{I_{PZ}} \end{bmatrix} \quad B = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ \frac{1}{aR_p^2 + I_{PY}} & 0 \\ 0 & 0 \\ 0 & \frac{1}{I_{PZ}} \end{bmatrix}$$

$$C = \begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix} \quad D = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$$

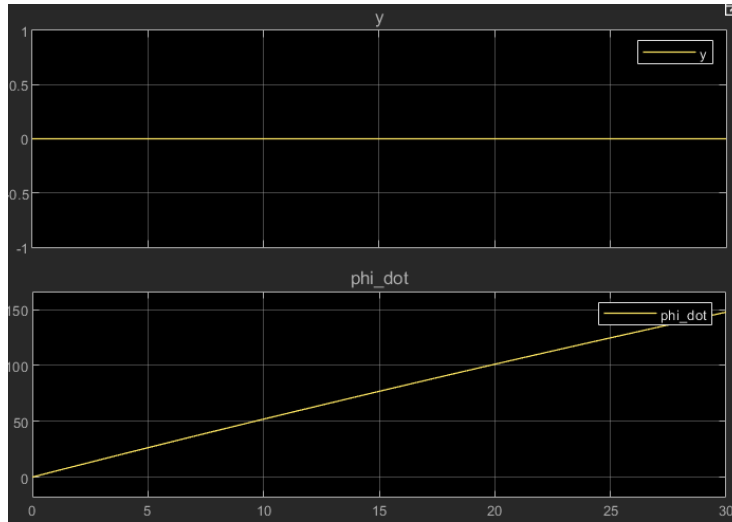
obtained by linearizing around the equilibrium state $\vec{z}_0 = [y \ \theta \ \dot{\varphi} \ \psi \ \dot{\psi}]^T = [0 \ 0 \ \frac{v_x}{R_p} \ 0 \ 0]^T$ and control $\vec{u}_0 = [\tau_\varphi \ \tau_\psi]^T = [c_\varphi \dot{\varphi} \ 0]^T$. We chose to observe the variables y and $\dot{\varphi}$ to achieve complete reachability and observability of the system.

4 Model implementation

The *Simulink* model in the figure contains both the linearized and nonlinear system dynamics:

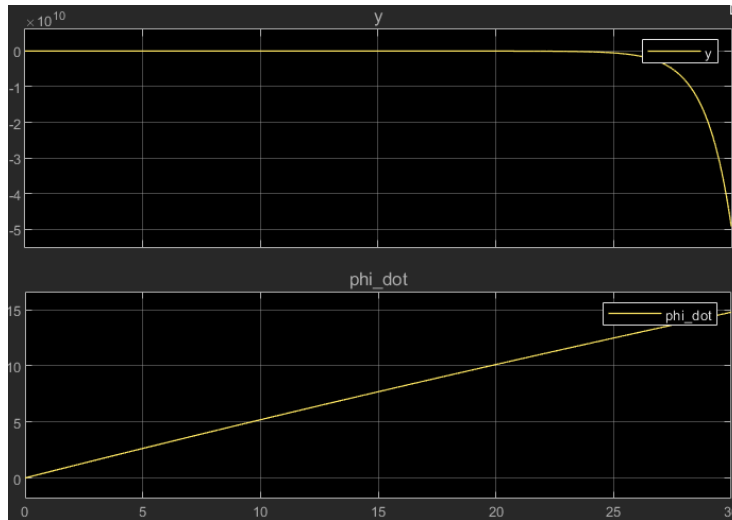
As expected, with zero torques the system remains stationary.

4.2 Input case $u = [10 \ 0]^T$



In the case just shown, $\dot{\varphi}$ increases while y remains constantly zero. These trends are consistent with the fact that the only nonzero torque is τ_{φ} .

4.3 Input case $u = [1 \ 0.01]^T$



In the case just shown, y decreases while $\dot{\varphi}$ increases. These trends are consistent with the input that was applied.

5 Modeling uncertainties on system parameters

To represent the uncertainty on the AGV parameters, the *ureal* command was used, which allows specifying for an uncertain parameter its nominal value and the range of values within which it can vary. In our case the two uncertain parameters are R_p and R_a , the rear and front wheel radii respectively. The system will therefore be associated with a diagonal uncertainty matrix Δ_G whose elements are the relative uncertainties of the two radii, with a number of occurrences equal to those of the uncertain parameters in the block diagram derived from the state-space form. To extract Δ_G , the *lftdata* command was used, which extracts from the system an 8×8 matrix Δ containing 4 occurrences of R_p and 4 of R_a . The same matrix could have been derived manually via the block diagram by correctly modeling each uncertainty.

6 Modeling uncertainties on actuators

To represent the uncertainties on the parameters of the actuator transfer functions, we chose a multiplicative input uncertainty model where each actuator has an uncertainty modeled independently from the other:

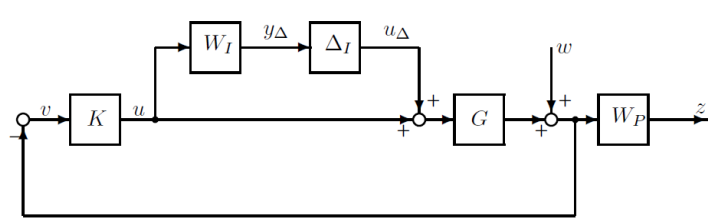


Figure 3: Multiplicative input model for the actuators

The matrix W_I will therefore be a 2×2 diagonal matrix containing the weights related to the uncertainties on the two inputs. To derive these weights, the Matlab function *ucover* was used, which extracts a weight function of the desired order from the nominal model and a sample of 100 instances of the perturbed transfer function. For example, for the torque τ_φ , the following code was used:

```

%% Actuator uncertainty and uncertain actuator models

orderWt1 = 2; % Weight order

% PHI
G_tau_phi = minreal(tf(K_m2, [T_m2^2/100 1.01*T_m2 1]));
G_tau_phi_nom = minreal(G_tau_phi.NominalValue);
G_tau_phi_nom_tf = minreal(tf(G_tau_phi_nom));

% Samples of G_tau_phi_array
rng('default');
G_tau_phi_array = usample(G_tau_phi, 100);

G_tau_phi_array_frd = frd(G_tau_phi_array, omega);
[usys_tau_phi, info_tau_phi] = ucover(G_tau_phi_array_frd, G_tau_phi_nom, orderWt1, 'InputMult');

WI_tau_phi_frd = info_tau_phi.Wlopt;
WI_tau_phi = tf(info_tau_phi.W1); % Actuator weight for tau_phi

```

The same procedure was applied for τ_ψ , but the nominal transfer function includes a delay which was modeled by considering the maximum value that the delay itself can assume.

The uncertain transfer function of each actuator is then obtained as follows:

$$G_{\tau_\varphi} = G_{\tau_{\varphi nom}} (1 + W_{I_{\tau_\varphi}} \Delta_I)$$

where Δ_I is a complex number with infinity norm less than 1, obtained via the *ultidyn* command. The actuator system is given by a diagonal transfer matrix containing the transfer functions just found. The overall system G_{tot} is then obtained by multiplying the AGV system by the actuator system. The block diagram of the system is therefore the following:

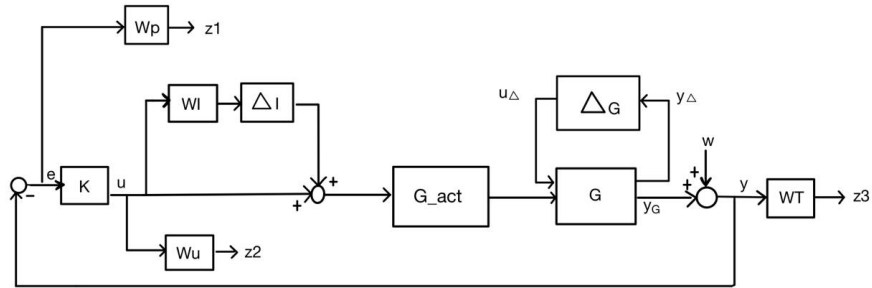


Figure 4: System block diagram

where W_p , W_u , and W_T are the weighting matrices that define the performance on the performance outputs z_1 , z_2 , and z_3 , which are respectively the error, the control action, and the output $y = [y \ \varphi]^T$. The choice of these weighting matrices will be explained in more detail in the H_∞ controller synthesis.

7 *Simulink* scheme for simulations

The *Simulink* scheme used to apply the various controllers and compare the linearized and nonlinear models is reported here:

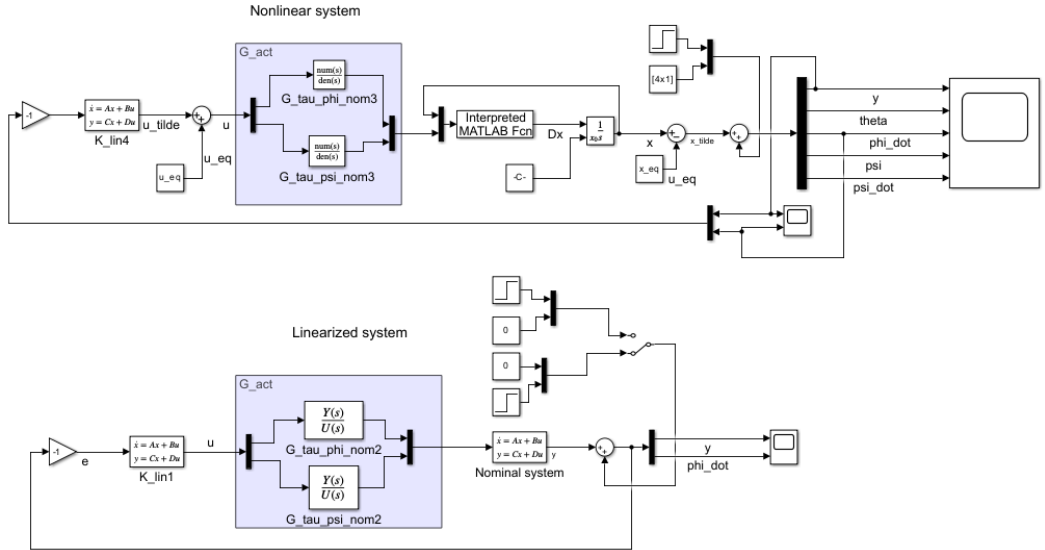


Figure 5: *Simulink* model for simulations

For the controller, a *state-space* block was used where the matrices A, B, C, and D are those provided by the synthesis scripts; to model the actuator transfer functions, two *Transfer function* blocks were implemented, one for each corresponding input, so as to obtain a diagonal matrix. The actuator and system parameters are the nominal ones.

8 LQG Controller

8.1 Traditional LQG Controller

Traditional LQG (*Linear Quadratic Gaussian*) control assumes that the system dynamics equations are linear and known, and that the measurement noise and process disturbances are stochastic with known statistical properties:

$$\begin{aligned}\dot{x} &= Ax + Bu + \Gamma w_d \\ y &= Cx + Du + w_n\end{aligned}$$

where w_d and w_n are zero-mean Gaussian stochastic processes, uncorrelated in time:

$$\begin{aligned}E[w_d(t)w_d(\tau)^T] &= W\delta(t - \tau) \geq 0 \\ E[w_n(t)w_n(\tau)^T] &= V\delta(t - \tau) > 0\end{aligned}$$

We further assume that w_d and w_n are also mutually uncorrelated.

The LQG control problem consists in finding the optimal control $u(t)$ that minimizes the cost functional:

$$J = E \left[\lim_{T \rightarrow \infty} \int_0^T (z^T Q z + u^T R u) dt \right]$$

where z is a linear combination of the states, while Q and R are constant, symmetric weighting matrices such that $Q \geq 0$ and $R > 0$.

The LQG control solution can be found through the separation principle:

1. The optimal estimate $\hat{x}$ of the state x is obtained by minimizing the estimation error covariance. This is achieved through the Kalman filter, whose gain matrix is K_f .
2. The estimate $\hat{x}$ obtained in the previous step is used as if it were an exact measurement of the state, to solve the linear quadratic control problem consisting of minimizing

$$\int_0^\infty (z^T Q z + u^T R u) dt$$

The solution assumes that u is a linear function of the state:

$$u = -K_c x$$

8.2 LQG Controller with integral action

Unlike the traditional LQG controller, the one with integral action requires that $G(s)$ be “augmented” with an integrator before the controller design itself. The integral action is subsequently brought back inside the controller.

8.3 Controller synthesis via the *lqg* command

The LQG controller can be implemented through the *lqg* command, which takes as input the nominal plant (composed of the AGV and actuators), and the matrices Q_{QR} , Q_{WV} , and Q_I , which we now define.

The matrix Q_{QR} is defined as follows:

$$Q_{QR} = \begin{bmatrix} Q & 0 \\ 0 & R \end{bmatrix}$$

where Q is the state weighting matrix and R is the input weighting matrix.

The matrix Q_{WV} is defined as follows:

$$Q_{WV} = \begin{bmatrix} W & 0 \\ 0 & V \end{bmatrix}$$

where W is the process disturbance covariance matrix and V is the measurement noise covariance matrix. Finally, the matrix Q_I weights the integral of the

tracking error (for the servo-controller case). This procedure can be used both for the 1 DoF case and for the 2 DoF case. Some results obtained for the LQG in *servo controller* mode using the Matlab *lqg* command are shown below:

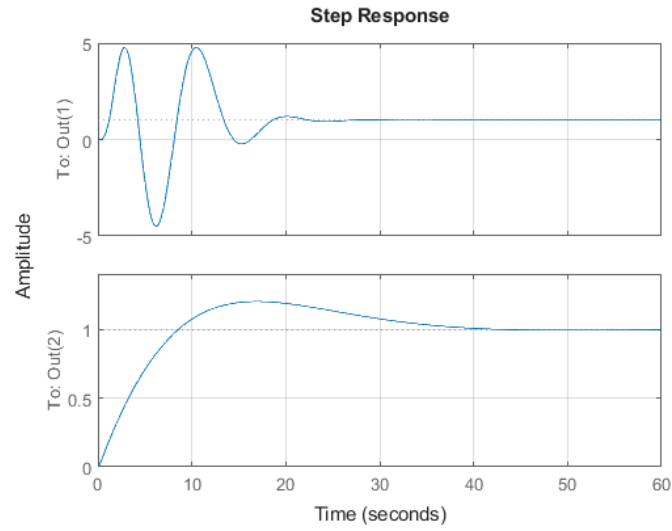


Figure 6: Step response plot for the 1 DoF controller

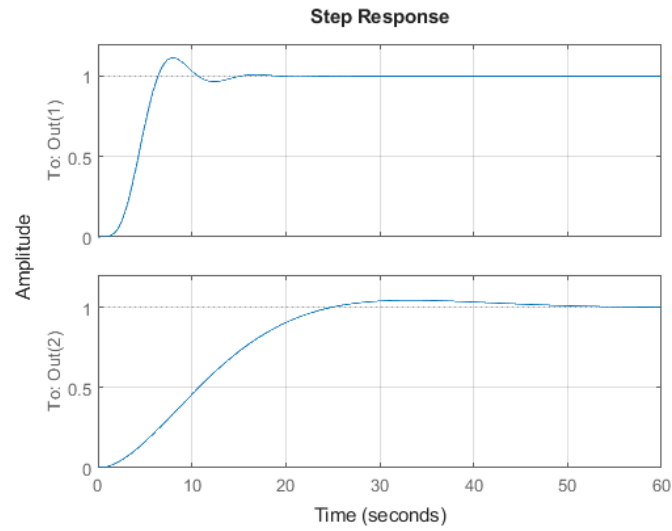


Figure 7: Step response plot for the 2 DoF controller

8.4 Controller synthesis via the *lqr*+*kalman* commands

The LQG controller can also be implemented through the two Matlab commands *lqr* and *kalman*. Specifically, an LQR controller is first found via the *lqr* command, which takes as input the matrices A and B of the nominal system along with the matrices Q and R representing the state and input weighting matrices respectively. Subsequently, the *kalman* command is used, which takes as input the system and the matrices W and V representing the process disturbance and measurement noise respectively.

This procedure can be used both for the 1 DoF case (controller from $-y$ to u) and for the 2 DoF case (controller from $[r \ y]^T$ to u).

Some results obtained for the LQG in *regulator* mode with integrator using the Matlab *lqr*+*kalman* commands are shown below.

The step response comparison plots between the 1 DoF and 2 DoF closed-loop controllers are the following:

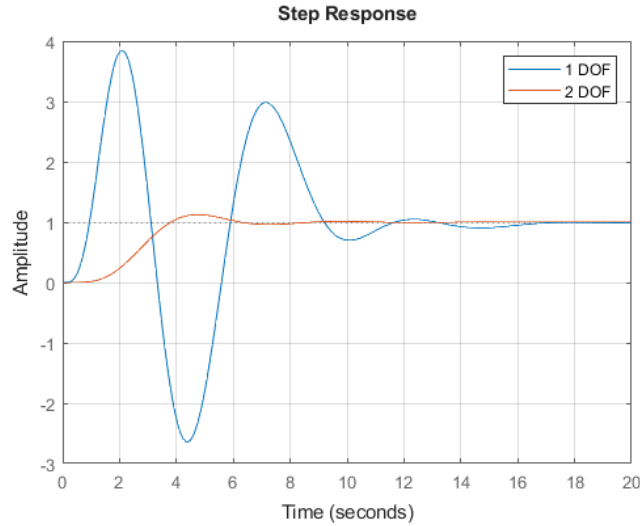


Figure 8: Step response on the first output

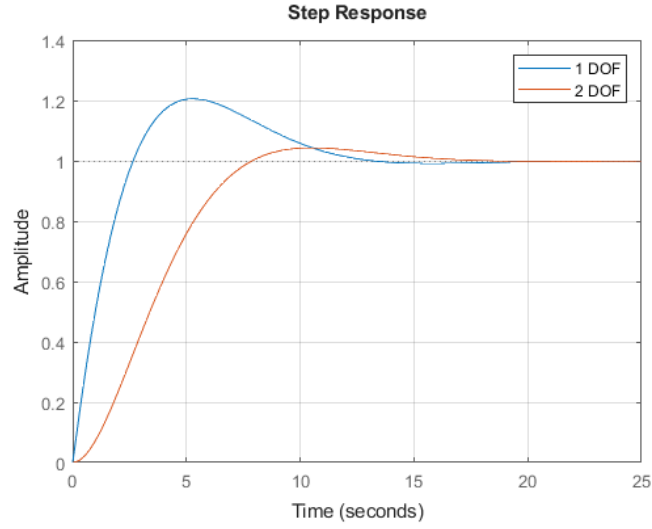


Figure 9: Step response on the second output

8.5 Application to the linearized system

Applying the 1 DoF *regulator* LQG controller to the linearized system and introducing a step disturbance, observing the behavior of y and $\dot{\varphi}$, the following results are obtained:

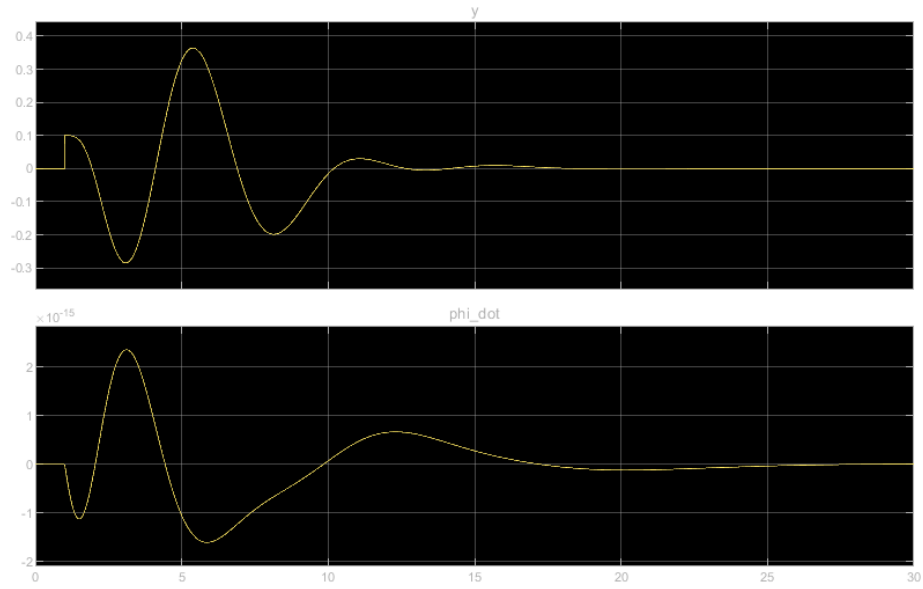


Figure 10: Unit step response on y

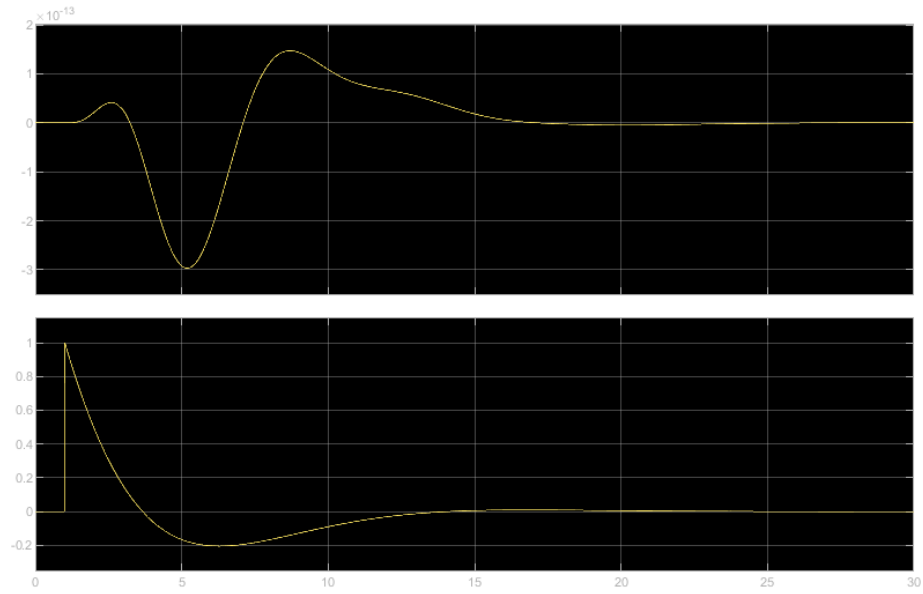


Figure 11: Unit step response on $\dot{\varphi}$

In general, the LQG controller yields satisfactory results with good conver-

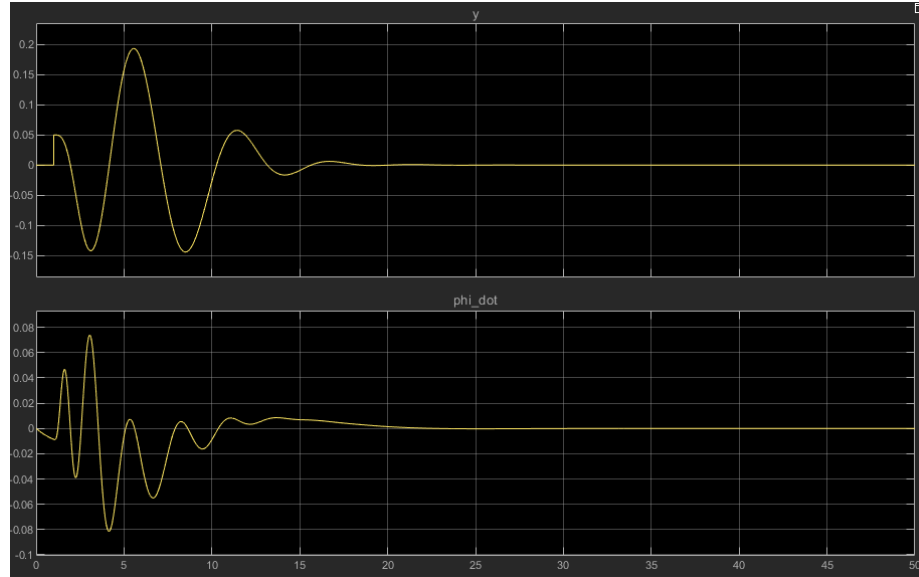
gence times for the error to reach zero (approximately 15–20 s). The same analysis was repeated for the various versions of the LQG controller and the results obtained are analogous to those just analyzed.

8.6 Application to the nonlinear system and RAS estimation

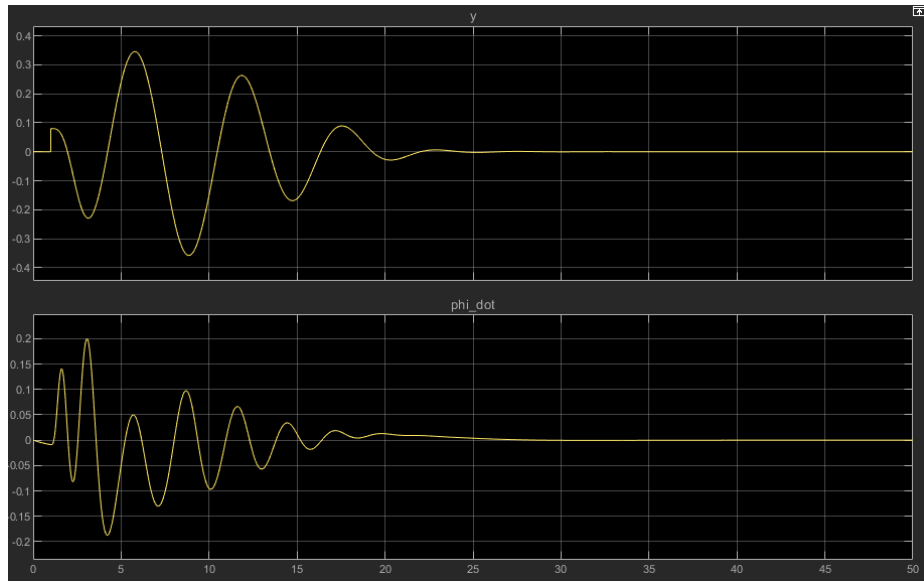
To empirically evaluate the Region of Asymptotic Stability (RAS) of the nonlinear system model, it was used the Simulink model describing it. The RAS is evaluated by varying the final value of the step disturbance until a value is found beyond which the system diverges. The controllers developed in this work are designed on the linearized system, so if the nonlinear system moves away from the equilibrium point it is likely that the controller will fail to prevent the system from diverging and thus departing from the desired equilibrium.

We therefore attempt to empirically compute the RAS by varying only the disturbance on y for simplicity.

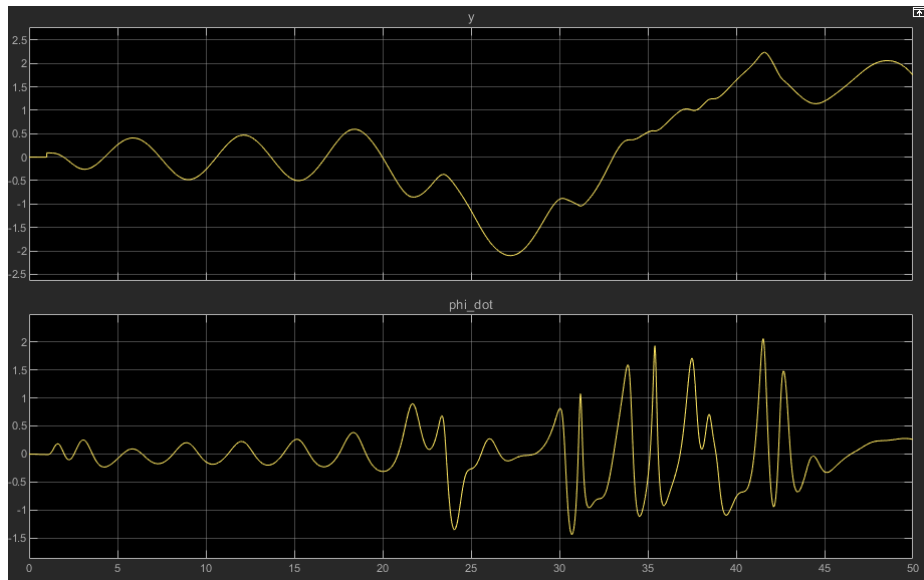
When the step has *final value* = 0.05, the following result is obtained:



When the step has *final value* = 0.08, the following result is obtained:



When the step has *final value* = 0.09, the following result is obtained:



From the analysis it follows that when the step has *final value* ≥ 0.09 , the nonlinear system diverges.

8.7 μ -analysis of the controller

To verify the robustness of the LQG controller, the μ -analysis is now performed, exploiting the structured singular value of the matrix N in the N - Δ configuration referred to a $\tilde{\Delta} = \begin{bmatrix} \Delta & 0 \\ 0 & \Delta_P \end{bmatrix}$ in which Δ is the matrix related to the actual uncertainties while Δ_P is a full matrix associated with performance and therefore a sort of “fictitious” uncertainty. The script used is the following:

```

%% Analisis of RP
%% Computation of mu for RP
blk = [10 10; 2 2]; % Full uncertainty structure
[mubnds_RP, muinfo_RP] = mussv(Nfr, blk);
muRP = mubnds_RP(:, 1);
[muRPinf, muRPw] = norm(muRP, inf);

```

```

%% Computation of mu for RS
blk = [10 10]; % True uncertainty structure
N11 = Nfr(1:10, 1:10); % extract N11
[mubnds_RS, muinfo_RS] = mussv(N11, blk);
muRS = mubnds_RS(:, 1);
[muRSinf, muRSw] = norm(muRS, inf);

```

```

%% Analisis of NP with mussv
Nnp = Nfr(11:12, 11:12); % extract N22
[mubnds_NP, muinfo_NP] = mussv(Nnp, [2 2]);
muNP = mubnds_NP(:, 1);
[muNPinf, muNPw] = norm(muNP, inf);

```

and it is divided into 3 parts:

1. In the first part, the structured singular value of the complete matrix N was computed by specifying the structure of $\tilde{\Delta}$ to the Matlab function *mussv*, and the maximum of the upper bound was then extracted via the infinity norm. This value is referred to as *muRPinf*.
2. In the second part, μ of the matrix N_{11} , consistent with the dimensions of Δ , was computed and the maximum was extracted as before, referred to as *muRSinf*.
3. In the last part, μ was computed for the matrix N_{22} , consistent with Δ_P , and its maximum was taken, referred to as *muNPinf*.

To verify each property, the upper bound plot of the respective μ must remain below unity. This procedure will also be applied for the μ -analysis of the H_∞

controller and the DK-iteration controller. The results of the μ -analysis of the LQG controller are the following:

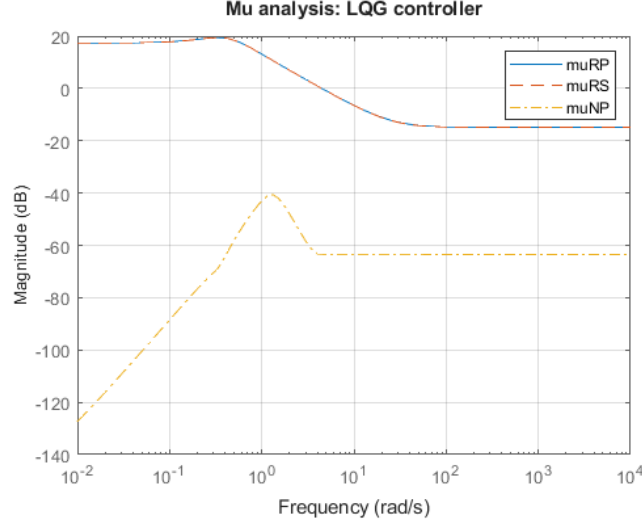


Figure 12: μ -analysis of the LQG controller

from which it can be seen that this controller is nominally performant (maximum value equal to 0.14) but not robust. Indeed, both robust stability and robust performance have peak values considerably above unity (9.37 and 9.39 respectively) over a certain frequency range. This means that robustness is guaranteed for only 11% of the uncertainty, i.e., for a value $\frac{1}{\mu}$. The possible solutions are therefore to reduce the uncertainty or the performance requirements. First, we reduced the ranges of the uncertain values, obtaining the following improvement:

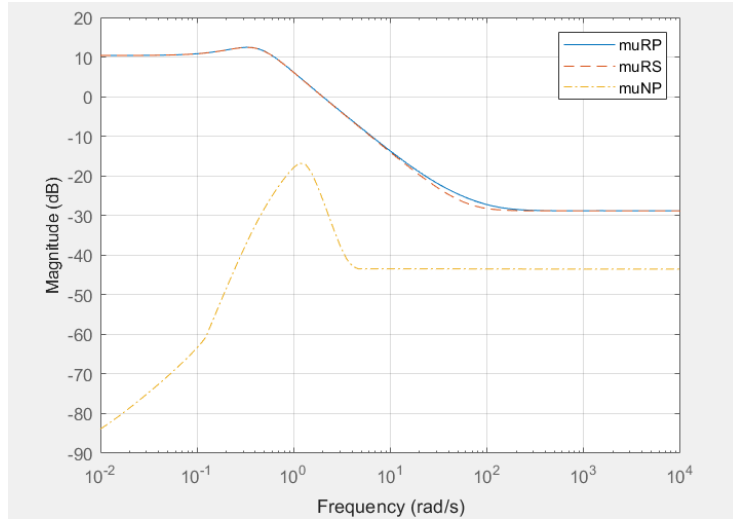


Figure 13: μ -analysis of the LQG controller with reduced uncertainty

from which a reduction in the upper bound of the structured singular values for RS and RP is visible. Further reducing the performance requirements yields an additional decrease as shown in the following figure:

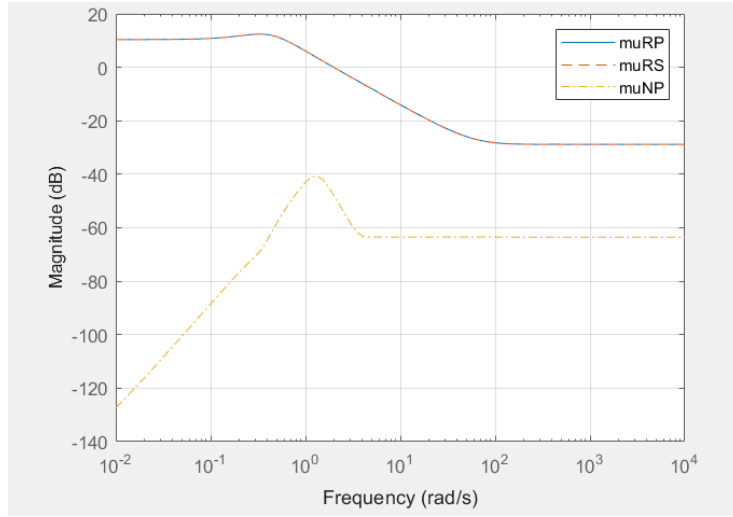


Figure 14: μ -analysis of the LQG controller with reduced uncertainty and performance

9 H_∞ Controller

The H_∞ synthesis was implemented using the *Mixed Sensitivity* technique, which allows designing a controller that satisfies given performance requirements on the sensitivity function S and on T , the complementary sensitivity function, through appropriate weighting functions. The H_∞ controller is therefore the result of minimizing the H_∞ norm of the matrix N defined as:

$$N = \begin{bmatrix} W_P S \\ W_U K S \\ W_T T \end{bmatrix}$$

related to the following standard scheme:

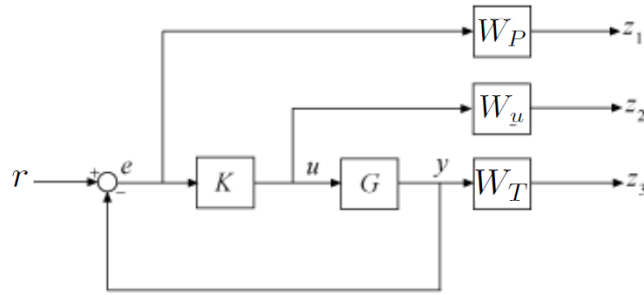


Figure 15: Standard scheme for *mixsyn*

In our case we performed a simplified analysis considering exclusively the error as performance output, therefore using only the weighting matrix W_P . The controller minimizes only:

$$N = [W_P S]$$

The weight W_P related to performance on the sensitivity function is a diagonal matrix whose entries have the following form:

$$W_{p_i} = \frac{s/M_i + \omega_{B_i}}{s + A_i \omega_{B_i}}$$

where ω_{B_i} is the bandwidth, M_i is the maximum amplitude peak of the sensitivity, and A_i is the maximum steady-state tracking error allowed.

9.1 Controller synthesis via the *mixsyn* command

The synthesis is performed through the Matlab *mixsyn* command, which derives a controller for the system in the standard form defined above; the results obtained are the following:

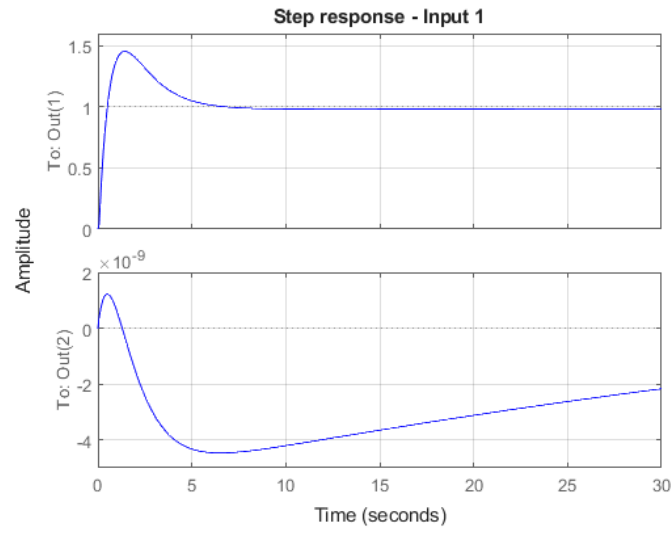


Figure 16: Step response on the first output

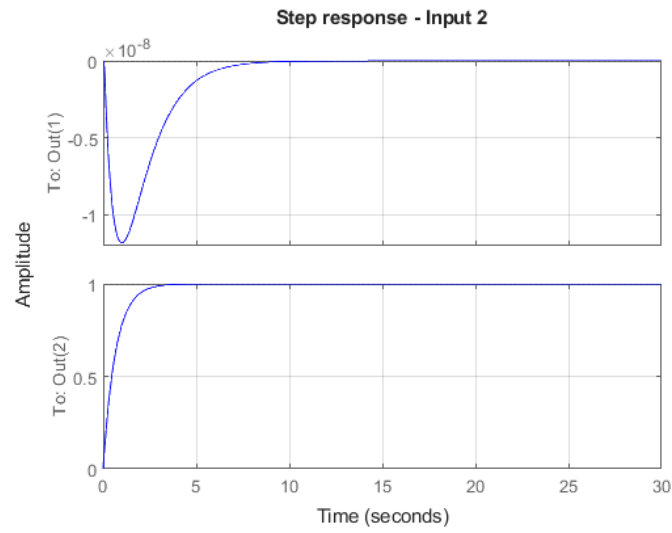


Figure 17: Step response on the second output

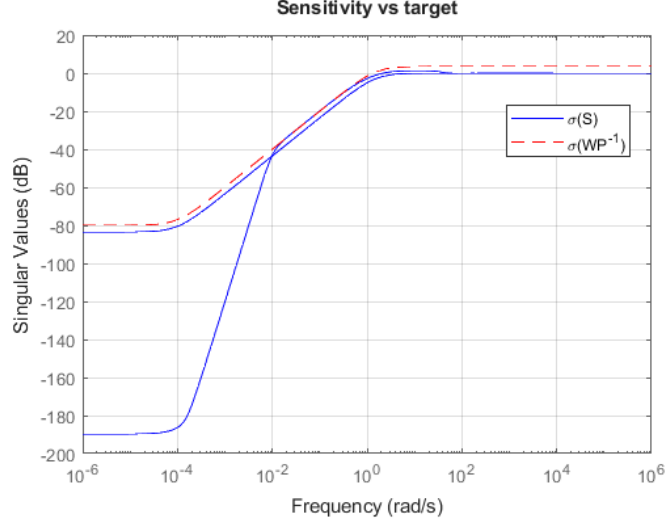


Figure 18: Comparison between S and $\frac{1}{W_P}$

In this last plot it can be observed that the singular value of the sensitivity function always stays below that of the inverse of the weighting function W_P , confirming that the performance requirements have been met.

9.2 Controller synthesis via the *hinfsyn* command

Another method for synthesizing the H_∞ controller uses the *hinfsyn* command, which is more general than *mixsyn* as it allows finding a controller that minimizes the H_∞ norm of N for a system of arbitrary form. This form must be specified via the *connect* command, which we will use later in the μ -analysis and which allows specifying the block structure of the system:

```

%% Generalized plant P by using

G.u = 'u';
G.y = 'yG';

WP_hinfsyn.u = 'e';
WP_hinfsyn.y = 'z1';

Sum1 = sumblk('y=w+yG',2);
Sum3 = sumblk('e= -y',2);

P_hinfsyn = connect(G, WP_hinfsyn, Sum1,Sum3,...
    {'w','u'},{'z1','e'});

[K_hinfsyn, CL_hinfsyn, gamma_hinfsyn, info] = hinfsyn(P_hinfsyn,2,2);

gamma_hinfsyn;    % Display peak value

```

The results of the *hinfsyn* command are similar to those of *mixsyn*, as expected:

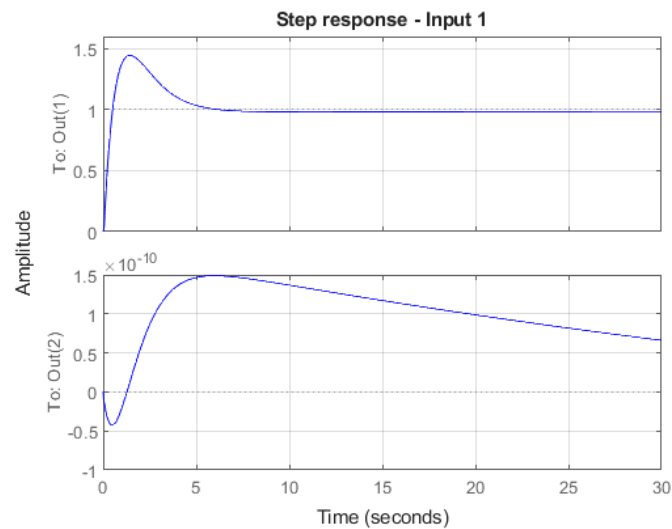


Figure 19: Step response on the first output

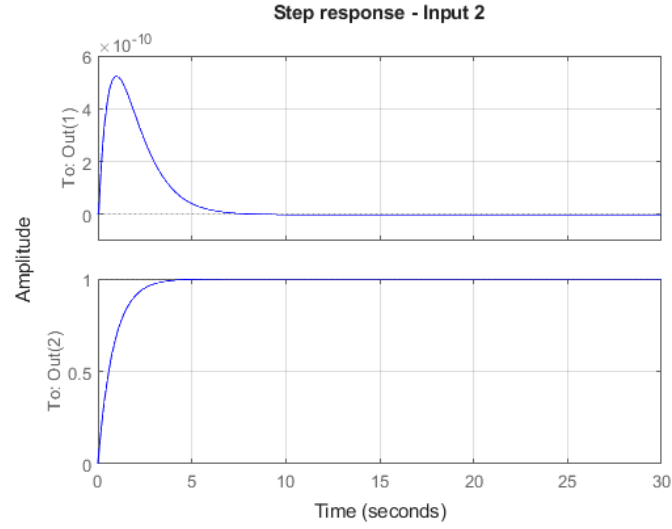


Figure 20: Step response on the second output

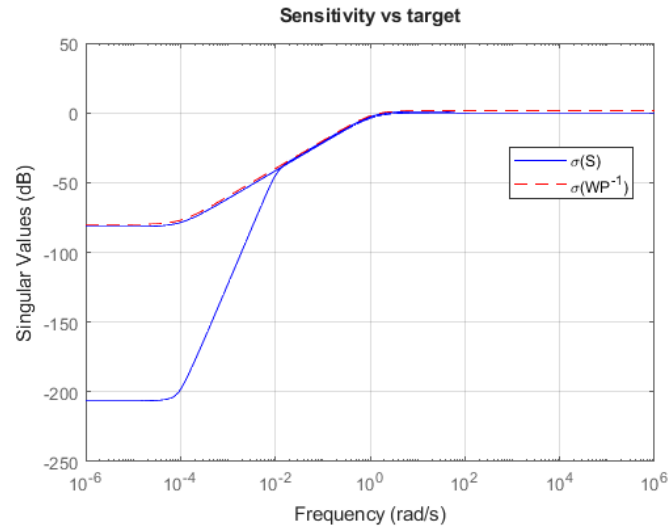


Figure 21: Comparison between S and $\frac{1}{W_P}$

9.3 Application to the linearized system

With the obtained H_∞ controller applied to the linearized system, the following plots are obtained:

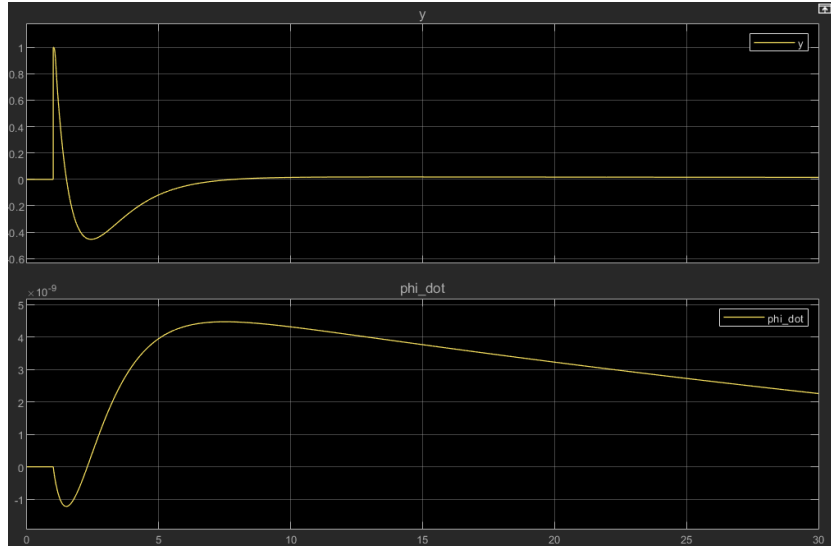


Figure 22: Unit step on the first output of the linearized system

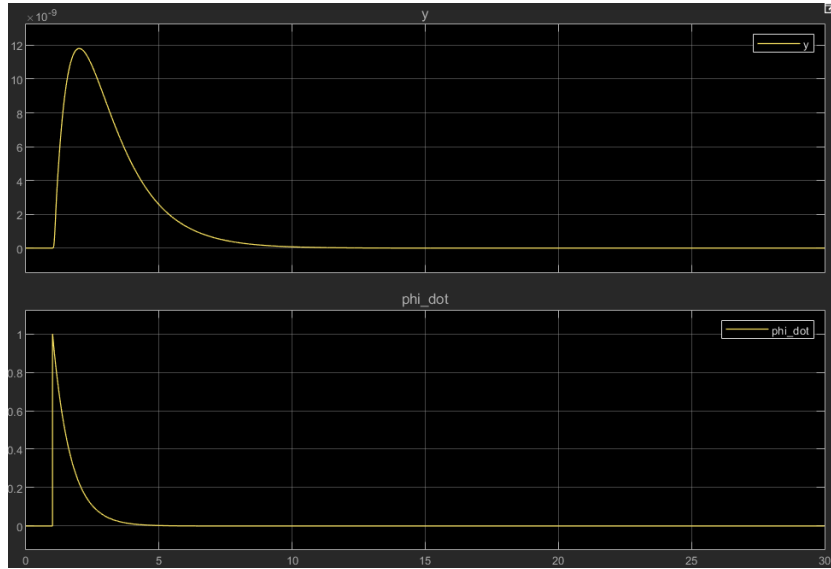


Figure 23: Unit step on the second output of the linearized system

9.4 Application to the nonlinear system and RAS estimation

Applying the controller to the nonlinear system instead, we obtain:

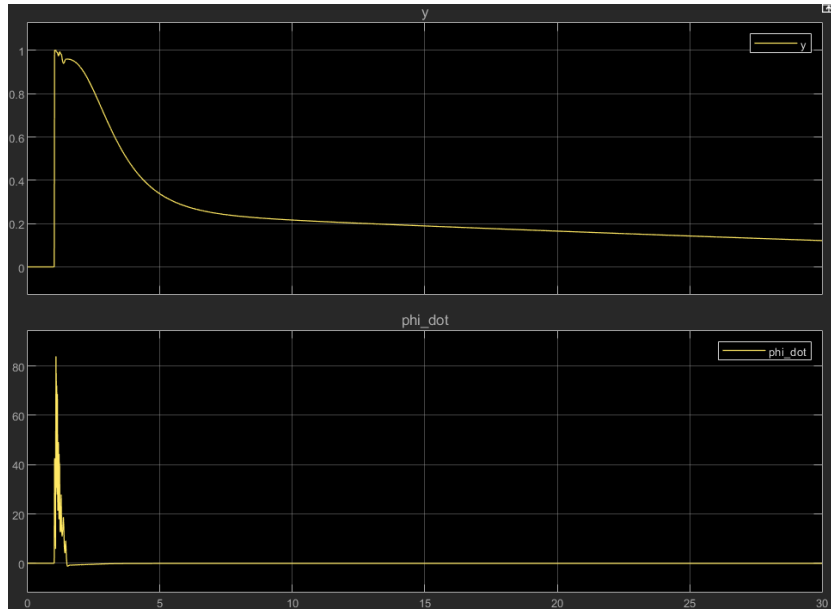


Figure 24: Unit step on the first output of the nonlinear system

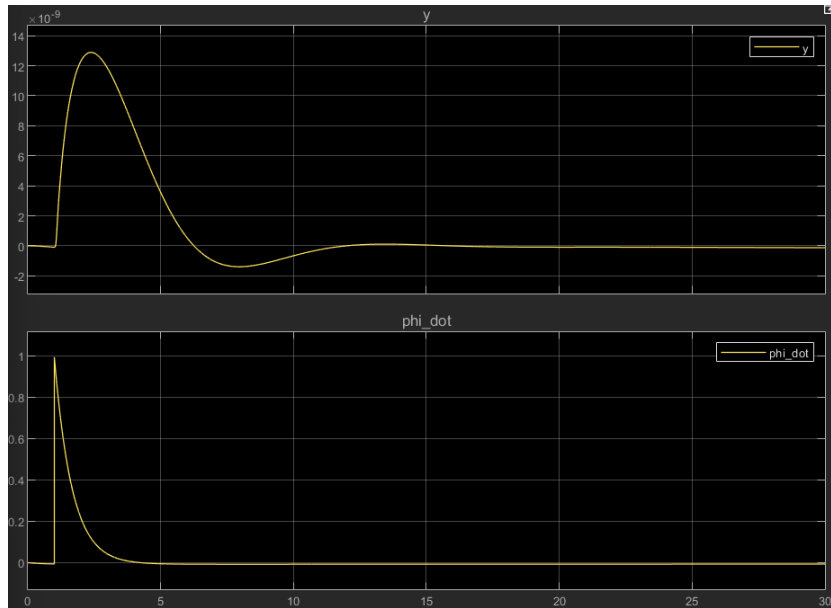


Figure 25: Unit step on the second output of the nonlinear system

For the RAS estimation, we start from a step value on the first input of 1

and increase it until convergence is lost:

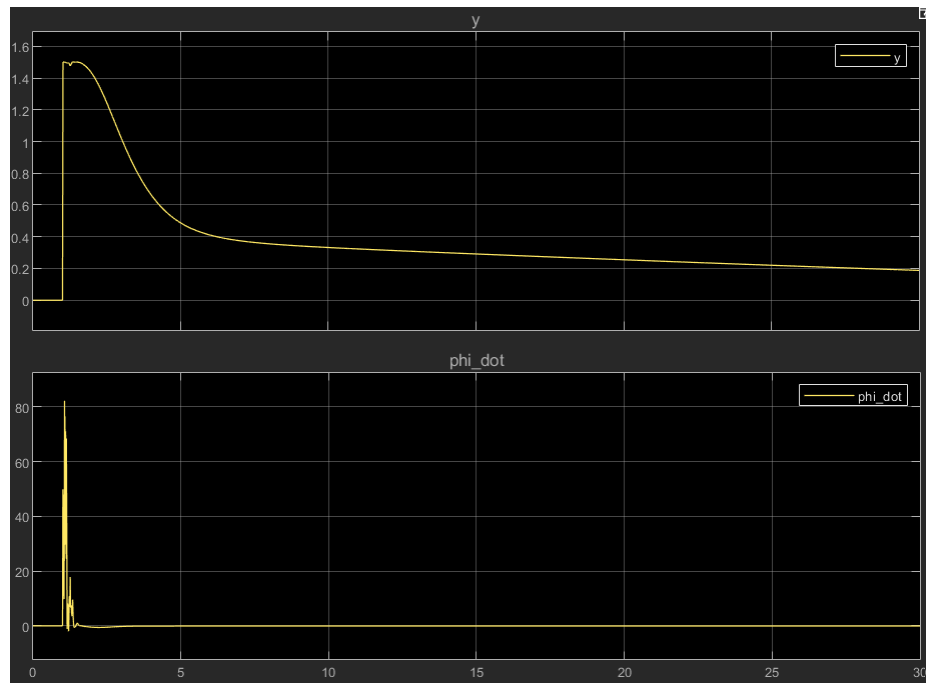


Figure 26: Step with final value 1.5

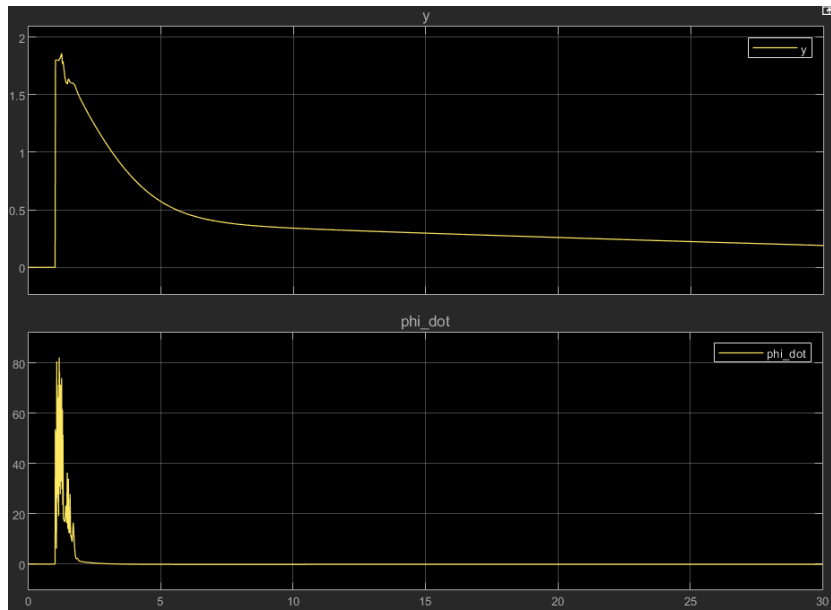


Figure 27: Step with final value 1.8

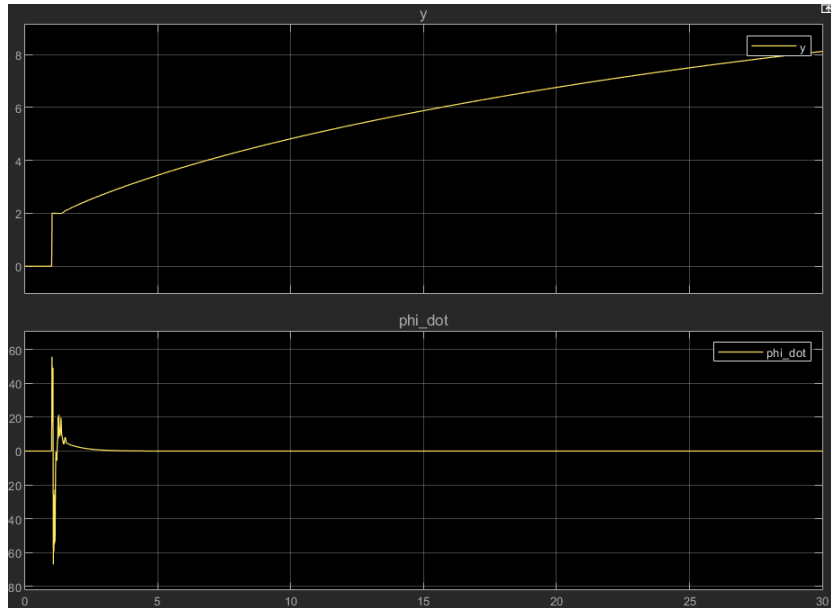


Figure 28: Step with final value 2

It can be seen that with a final value of 2, y diverges and we are therefore

outside the RAS.

9.5 μ -analysis of the controller

Using the same procedure as for the LQG controller, the μ -analysis of the H_∞ controller is now performed; the results are the following:

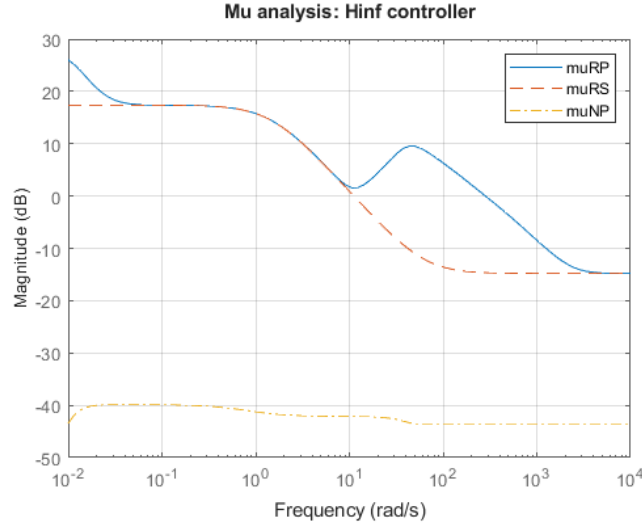


Figure 29: μ -analysis of the H_∞ controller

from which it can be noted that the H_∞ controller is also not robust, since the values of μ_{RP} and μ_{RS} exceed unity (20.04 and 7.38 respectively), meaning that robust stability is guaranteed for only 14% of the uncertainty and robust performance for only 5%. Reducing both the uncertainty values and the performance requirements in this case as well yields the following results:

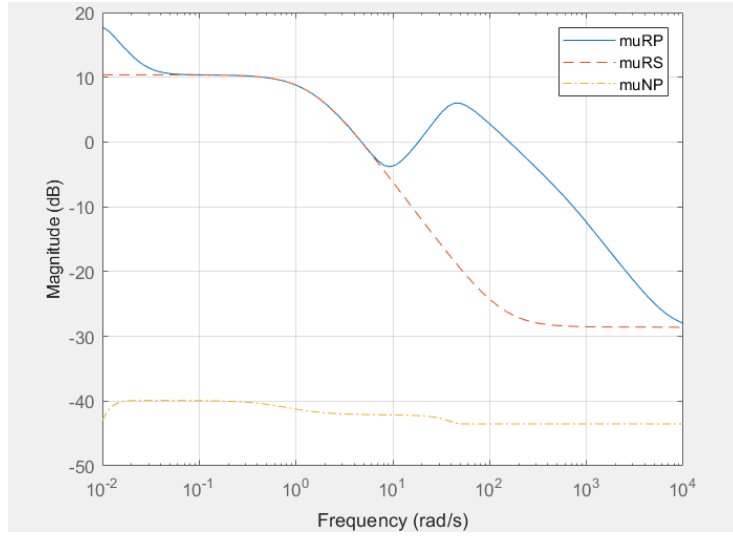


Figure 30: μ -analysis of the H_∞ controller with reduced uncertainty

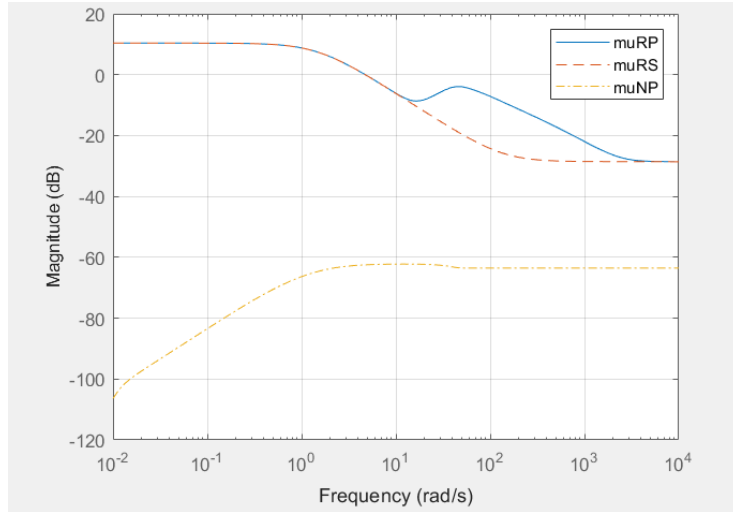


Figure 31: μ -analysis of the H_∞ controller with reduced uncertainty and performance

10 DK-iteration Controller

To date, no direct method exists for designing an optimal μ controller. For this reason, the DK-iteration is implemented, a technique that combines μ -analysis and H_∞ controller synthesis. The basic idea of this technique is to find a

controller that minimizes the following upper bound of the structured singular value:

$$\mu(N) \leq \min_{D \in \mathcal{D}} \bar{\sigma}(DND^{-1})$$

where each matrix $D \in \mathcal{D}$ is real positive and has the form $D = \text{diag}\{d_i I_i\}$, with d_i scalar. The dimension of each block (i.e., the dimension of I_i) equals the dimension of the corresponding block Δ_i .

We therefore seek a controller that minimizes the maximum peak of this upper bound, i.e.,

$$\min_K (\min_{D \in \mathcal{D}} \|\mathbf{DN}(\mathbf{K})\mathbf{D}^{-1}\|_\infty)$$

alternating the minimization of $\|\mathbf{DN}(\mathbf{K})\mathbf{D}^{-1}\|_\infty$ with respect to K and with respect to D (keeping the other matrix fixed).

The DK-iteration consists of three steps:

1. **K-step:** synthesis of an H_∞ controller, $\min_K \|\mathbf{DN}(\mathbf{K})\mathbf{D}^{-1}\|_\infty$, keeping the matrix D fixed.
2. **D-step:** finding $D(j\omega)$ that minimizes $\bar{\sigma}(DND^{-1})$ at each frequency, keeping the matrix N fixed.
3. **Fitting step:** fitting the magnitude of each element of $D(j\omega)$ with a minimum-phase transfer function $D(s)$. The algorithm then returns to step 1.

10.1 Controller synthesis via DK-iteration

To implement the method just introduced in Matlab, we considered the nominal system and the following matrices:

- the uncertainty matrix Δ_I (already discussed) which allows modeling the uncertainty as multiplicative at the input,
- the weighting matrix W_P acting on the sensitivity function S .

Subsequently, the generalized plant P with multiplicative uncertainty was built using the Matlab *connect* command.

The matrix D was initialized to the identity matrix and the actual implementation of the controller via DK-iteration then followed. The individual steps are described below:

1. For the first step, a scaled (D matrix) H_∞ controller synthesis was performed using the *hinfsyn* command. Subsequently, the lower LFT was computed to find the matrix N :

```
% STEP 1: find an optimal H-infinity controller
[K, Nsc, gamma, info] = hinfsyn(D*P/D, nmeas, nu, 'method', 'lmi', 'Tolgam', 1e-3);

Nf = frd(lft(P,K),omega);
```

2. μ was computed by exploiting its upper bound discussed previously, using the Matlab *mussv* command. This command takes as input the matrix N found in the previous step and the matrix *blk* describing the uncertainty structure:

```
% STEP 2: compute mu using the upper bound
[mubnds,Info] = mussv(Nf, blk);
bodemag(mubnds(1,1), omega);
murp = norm(mubnds(1,1), inf,1e-6);

legendEntries{end+1} = sprintf('\mu at step %d', k);
```

3. The fit between $D(j\omega)$ found in the previous step and a fourth-order minimum-phase transfer function was performed using the *fitmagfrd* command. The matrix D is then updated based on the fit using the *append* command.

```
% STEP 3: fit the derived D
[VDelta,VSigma] = mussvextract(Info);
VSigma.DLeft = VSigma.DLeft/VSigma.DLeft(3,3);
d1 = fitmagfrd(VSigma.DLeft(1,1),4); % fit 4th order minimum phase
hold on
grid on
```

The result of the algorithm is the following:

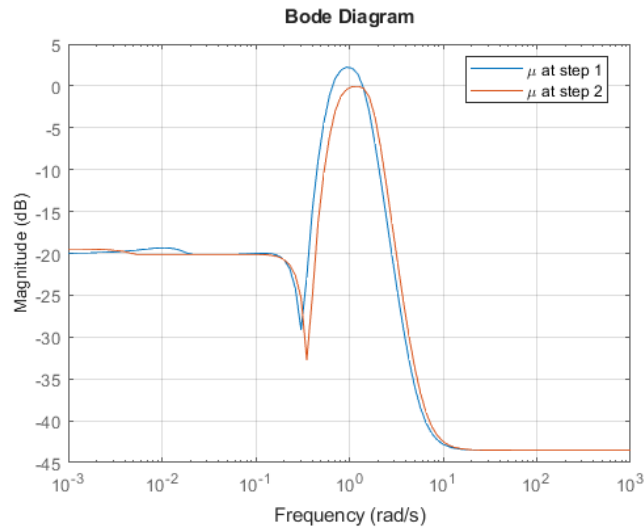


Figure 32: Behavior of the structured singular value across the iteration steps

Two iterations were performed in the script, since satisfactory results were

already obtained with this number. Indeed, the structured singular value at the second step is less than 1.

10.2 Application to the linearized system

The controller derived with the DK-iteration was then applied to the linearized *Simulink* system, obtaining the following results:

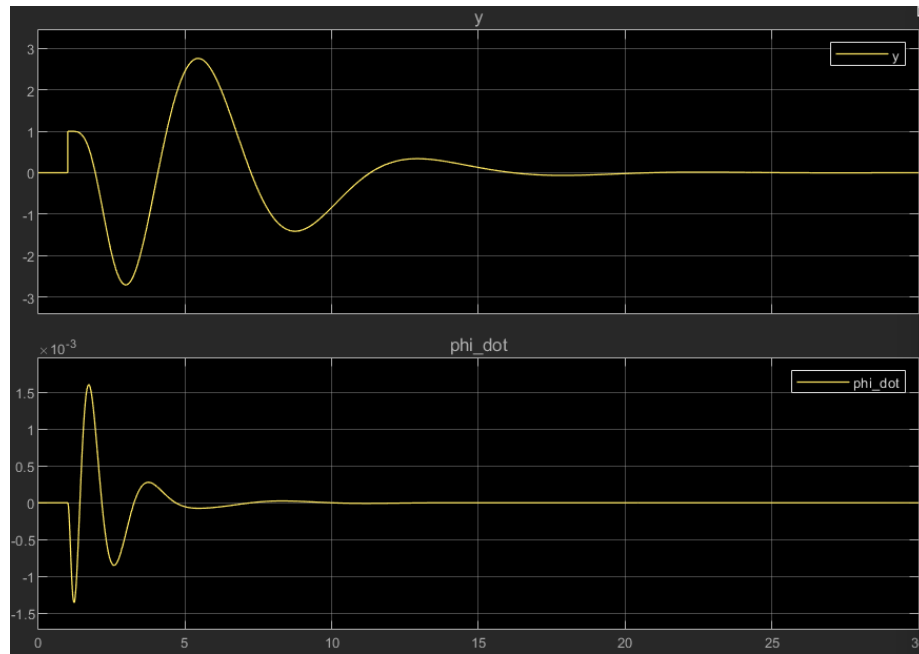


Figure 33: Step on the first output of the linearized system

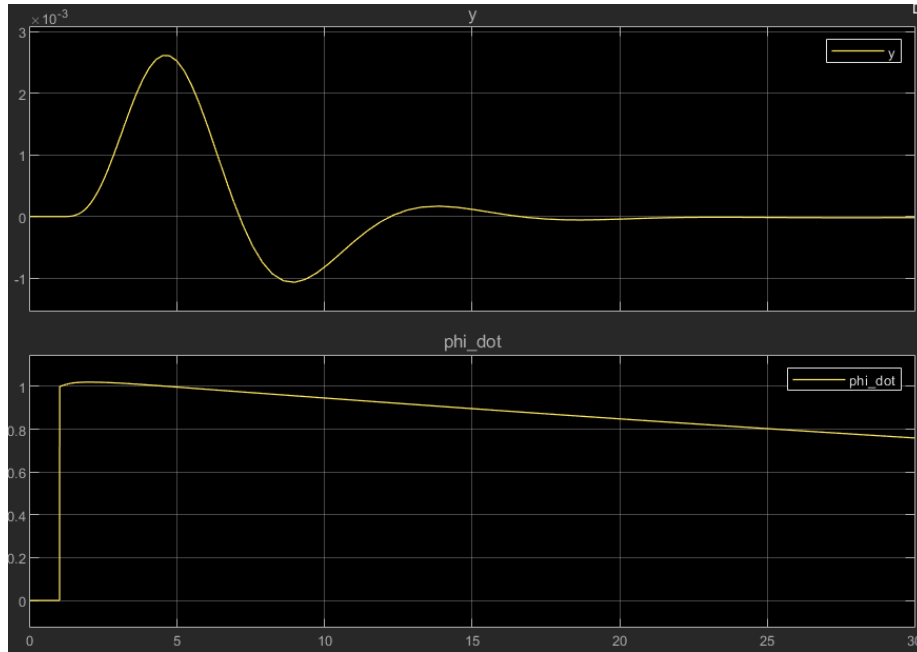


Figure 34: Step on the second output of the linearized system

10.3 Application to the nonlinear system and RAS estimation

When applying the controller to the nonlinear system, the following issue was encountered: with performance requirements tight enough to synthesize a robust controller (keeping the structured singular value below unity as in the previous figure) the *Simulink* simulation shows highly oscillatory behavior:

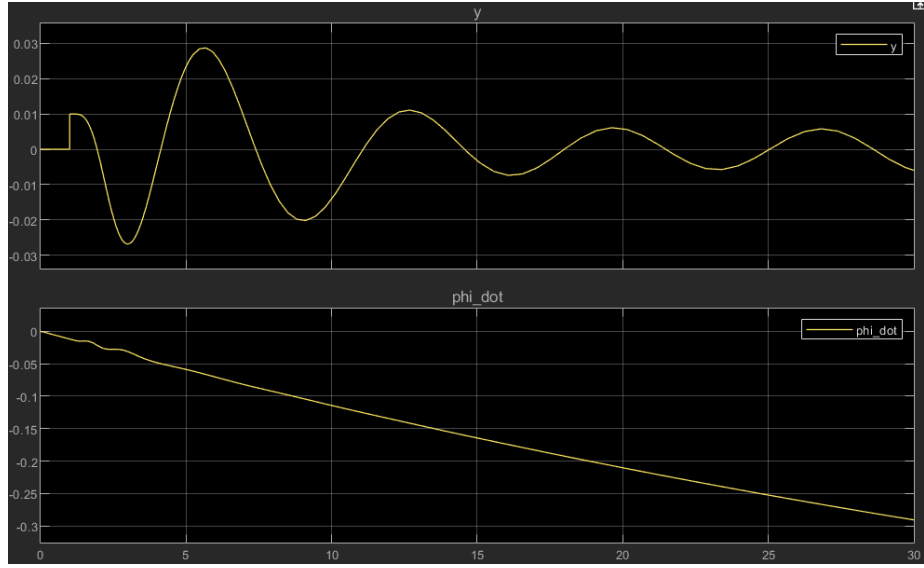


Figure 35: Step of 0.01 on the first output of the nonlinear system

in which $\dot{\varphi}$ does not converge. By instead requiring better (less stringent) performance, the controller turns out to be less robust, with a maximum structured singular value of 10 dB over a small frequency range (between 0.8 rad/s and 2.4 rad/s), but $\dot{\varphi}$ manages to converge, albeit over a long time:

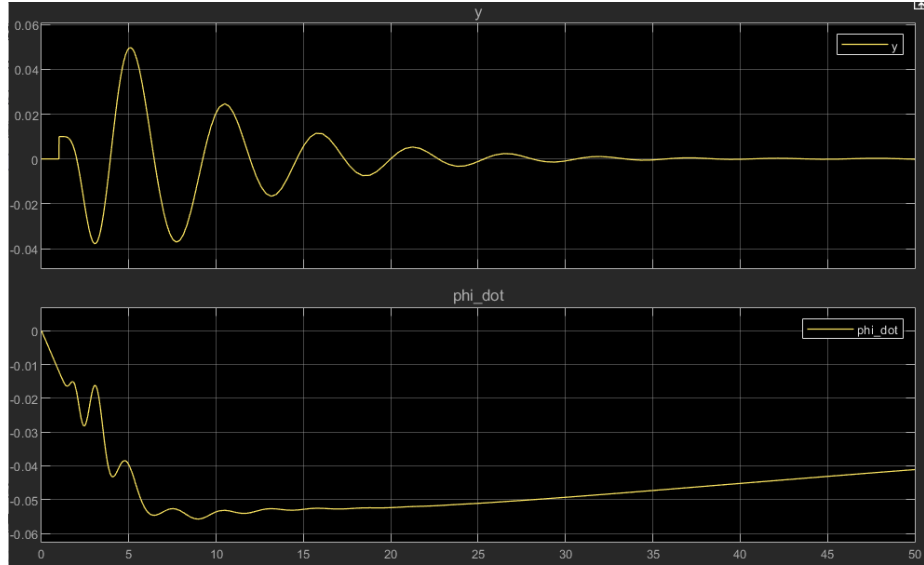


Figure 36: Step of 0.01 on the nonlinear system for the less robust controller

To estimate the region of asymptotic stability, as done previously, we increased the step value on the first output of the disturbance until divergence occurs:

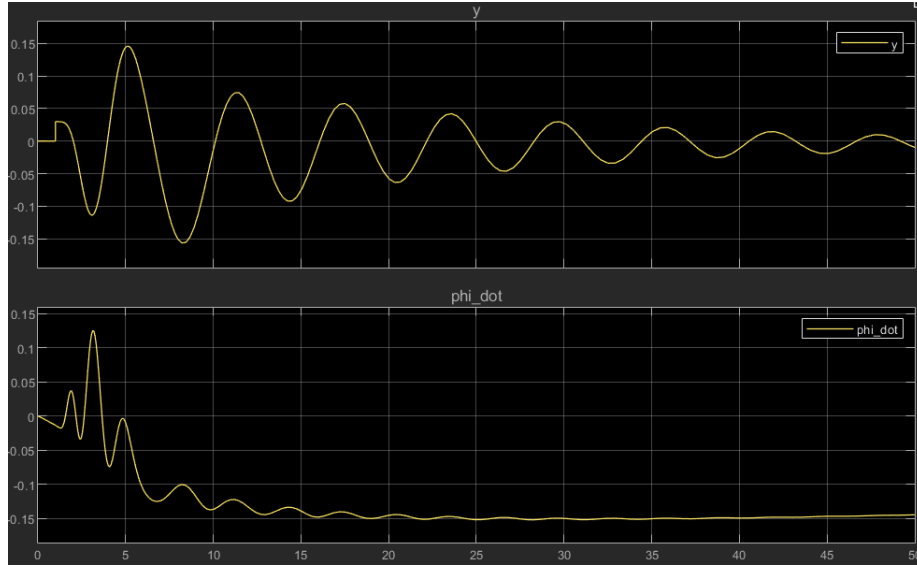


Figure 37: Step on the first output with final value 0.03

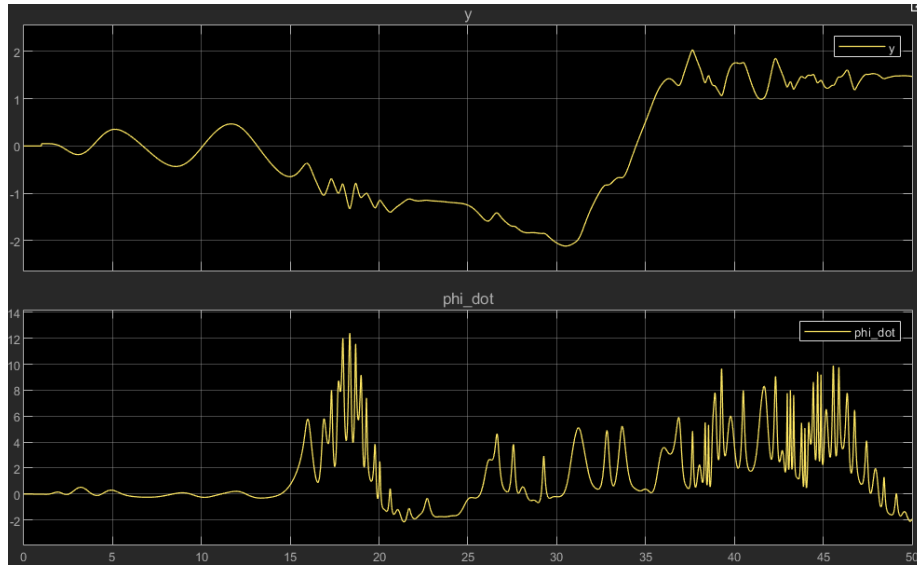


Figure 38: Step on the first output with final value 0.05

For a step amplitude greater than 0.05, the nonlinear system therefore exhibits divergent behavior.

10.4 μ -analysis of the controller

Applying the μ -analysis also to the controller synthesized with the DK-iteration, a marked improvement in robustness is evident (except in the very low frequency range), which is consistent with the fact that the DK-iteration is based precisely on the structured singular value:

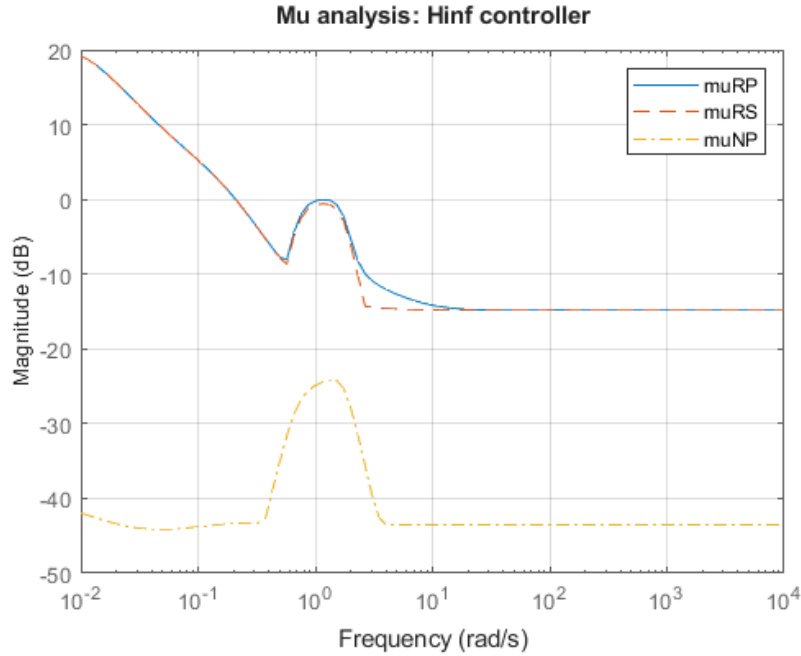


Figure 39: μ -analysis of the DK-iteration controller

11 Conclusions

As a result of our analysis, the controllers synthesized with the LQG and H_∞ techniques produce good results for the nominal system, both in the linearized and nonlinear cases, provided the system remains close to the equilibrium point. However, the robustness analysis has shown that neither controller is robust under those uncertainty values and performance requirements, which makes it necessary to reduce one or the other. For the synthesis of a robust controller, the DK-iteration technique is therefore employed, which yields a robust controller by alternating μ -analysis and H_∞ synthesis. The μ -analysis of the latter indeed satisfies all the imposed robustness constraints.